

机制内存学

Mechanical Sympathy:代码功能对了之后,如何再快一个数量级

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要。无锁那一座山解决的是“正确”——代码在并发下不出错。这一座山解决的是“快”——同样正确的代码,如何再快一个数量级。分水岭不在算法复杂度,而在 CPU 与内存之间那道内存墙:CPU 比内存快约 100 倍,大部分时间在等内存,不在算。本报告从五个可实测的现象出发——false sharing(独立变量为何互相拖累)、分支预测(排序数组为何处理更快)、cache 局部性(顺序访问为何快一个数量级)、AoS vs SoA(数据布局如何决定 cache 有效带宽)、内存墙与 prefetch(为何 pointer chasing 是性能杀手)——每一个都测量数字、解释机制。全程强调一个纪律:这些微架构惩罚都是“几倍到几十倍”的量级,不是百万倍;而且它们的惩罚机制彼此不同(cache invalidate vs pipeline flush vs cache miss),绝不能混为一谈。最后收束到一条主线:HFT 偏爱数组预分配、循环复用、热路径不 new,不是玄学,而是因为数组的地址可预测,prefetch 与内存级并行能翻越内存墙;链表的 pointer chasing 让内存延迟完全裸露,翻不过去。

0. 三个必须分清的“惩罚”——贯穿全文的纪律

在讲五个现象之前,先立一条纪律,因为它是最容易犯错的地方:不同现象的性能惩罚,机制完全不同,不能互相套用。一旦把一个现象的词套到另一个上,分析就全错。

三个正交的惩罚,务必分清。cache invalidate(缓存失效):属于多核一致性。别的核改了同一条 cache line,我这份副本作废——这是 false sharing 的机制,单线程不会发生。pipeline flush(流水线冲刷):属于分支预测。CPU 投机执行猜错了分支,把错误路径的指令全部丢弃、重填流水线——单核就发生,与 cache 无关。cache miss(缓存未命中):属于访存局部性。要的数据不在 cache,得从内存搬——单核发生,与多核一致性无关。三者延迟量级也不同:一次 cache miss ~100–300 周期,一次 pipeline flush ~15–20 周期,而 cache hit ~4 周期。

量级纪律:几倍到几十倍,不是百万倍。所有微架构惩罚都是常数倍的放大:一次 miss 比一次 hit 慢几十倍,一次 flush 比一次正确预测多十几周期。它们叠加起来让程序慢几倍到几十倍,绝不会是百万、千万倍。任何推导若得出“百万倍”,一定是把“单次惩罚”误当成了“整个操作重来无数次”。记住这条,能挡掉一大类量级错误。

1. False Sharing:独立变量为何互相拖累

两个线程各自狂加一个计数器,各写各的、逻辑上完全独立、不共享任何变量。把两个计数器紧挨着放(大概率同一条 64 字节 cache line)对比用 padding 隔到不同 line,在多核上前者会慢数倍。这是最反直觉的一题:明明不共享,为何互相拖累?

机制:一致性协议的粒度粗于数据冲突的粒度。CPU 读写内存以整条 cache line(64 字节)为单位,核间一致性(MESI)也以整条 line 为单位。两个计数器挤在同一条 line 上,虽然两个核改的是 line 里不同的字节(逻辑不冲突),但 MESI 分不清这一点——它只看到“两个核都想独占同一条 line”。于是核1 写时要独占这条 line、使核2 的副本失效;核2 写时又使核1 的失效——这条 line 的独占权在两核缓存间反复横跳(ping-pong),每次横跳都要跨核传输整条 line。共享是假的(改的是不同字节),争用是真的(同一条 line)——故名 false sharing。

几个要点。其一,单核不会发生 false sharing:任一时刻只有一个核在写,那条 line 无需跨核同步,挤不挤在一起都无所谓。它是纯粹的多核现象。其二,惩罚随核数增加而加重:抢同一条 line 的核越多,ping-pong 越惨烈。其三,解法极简——alignas(64) 或手动 padding,把互相独立、各线程各写的变量推到不同 cache line;C++17 的 hardware_destructive_interference_size 就是为此而设。

注意别把机制说错:这里没有锁,不是“一个线程等另一个的锁”;是 MESI 一致性协议强制两核轮流独占同一条 line、反复使对方副本失效。是硬件层面的 cache line 争夺,不是软件锁。

2. 分支预测:排序数组为何处理更快

一个装满随机数(0-255)的数组,循环里做 `if (arr[i] >= 128) sum += arr[i];`。把数组排序后再跑同样的循环,比乱序快数倍——尽管两者做的算术完全一样(同样的比较、同样多次加法)。唯一区别是数据有没有排序。

机制:分支预测 + 流水线 flush。CPU 是深流水线执行的,遇到 if 分支时不知道该走哪条路,但不能停下来等——它会投机(speculate):猜一个方向,提前把猜测路径的后续指令送进流水线。猜对:流水线满载,零浪费。猜错:已投机执行的指令全是错的,必须flush 流水线(丢弃、重填),约 15-20 周期。排序后,前半段全 <128、后半段全 ≥128,分支结果连续一大段相同,预测器几乎全中,极少 flush;乱序时结果随机,预测器蒙不中,约 50% 迭代都要 flush。

量级账要算清(这是加法不是乘法):设无 mispredict 的单次迭代成本 $\approx C$ 周期(约 3-5),一次 mispredict 惩罚 $\approx P$ 周期(约 15-20)。排序 $\approx 100万 \times C$;乱序 $\approx 100万 \times C + 50万 \times P$ ——乱序是在排序的基础上额外加上 50 万次 flush 的开销,不是把每次计算乘以某倍数。实测比值约 3-6 倍(循环体越轻,C 越小,flush 占比越大,比值越高)。关键洞察:flush(~18 周期)远大于单次计算(~4 周期),所以每次 flush 都很贵——正因为它比计算贵好几倍、又发生几十万次,才造成数倍差距。

实测中的意外:编译器会自动 branchless 化(CMOV)。在 -O2 下直接编译上面的循环,排序/乱序差距可能只有 ~1.25 倍——因为编译器把 if 编译成了条件传送指令 CMOV:两条路的结果都无条件算出来,再按条件选择性写回,没有跳转 → 没有预测 → 没有 mispredict → 没有 flush。要看到真正的分支惩罚,须用编译器无法 branchless 化的写法(如分支体内含无法内联的函数调用),此时实测约 3.2 倍。真实世界的一课:你以为的分支惩罚,常已被编译器用 CMOV 悄悄消除;你手写 branchless 的收益,很多时候编译器已替你拿了。真正还痛的,是编译器无法 branchless 化的分支(分支体有函数调用、复杂副作用、内存写入)。

3. Cache 局部性:顺序访问为何快一个数量级

遍历一个远大于 L3 的大数组(如 256MB,L3 仅 18MB),累加所有元素。顺序访问 `arr[0],arr[1],...` 对比随机下标访问——读的元素完全一样、加法次数完全一样,只是访问顺序不同。顺序快一个数量级。

机制:cache line 有效密度 + prefetch + 内存级并行(MLP)。其一,cache 一次搬整条 64 字节。顺序访问:每搬一条 line(16 个 int),后续 15 个都命中,miss 率 $\sim 1/16$ 。随机访问:几乎每次跳到不同 line,miss 率 ~ 1 。其二,硬件 prefetcher 能预测有规律的地址(等差 $base+i*4$),提前把后续 line 搬进来——顺序访问的 miss 被大量隐藏;随机访问地址无规律,prefetch 失效。其三,MLP:顺序访问地址都能提前算出,CPU 可同时发起多个 miss 让延迟重叠;随机访问同样能算出地址(若是下标随机而非 pointer chasing)故 MLP 仍部分有效,但 prefetch 无效。综合下来实测常在 5–30 倍。

量级要拧准。一次 cache miss 打到 DRAM $\sim 100\text{--}300$ 周期,一次 L1 hit ~ 4 周期,单次 miss/hit 比 ~ 75 倍。但总时间倍数不等于 75:顺序版并非纯 hit(带 $1/16$ 摊薄的 miss + prefetch 加持),分母不是 4;随机版有 MLP 重叠,分子也没那么惨。故总倍数落在 5–30 之间,不是 75。若刻意构造“每次都跳到很远的新页”,还会叠加下面的 TLB miss,把倍数推向高端。

第二套缓存:TLB(地址翻译缓存),别与数据 cache 混。程序用虚拟地址,数据在物理地址,中间靠页表(存在内存里)翻译。每次访存都要翻译,太慢,故 CPU 用 TLB 缓存最近的“虚拟页 $\rightarrow$ 物理页”映射。TLB miss = 翻译不在 TLB,须去内存走页表(page walk),几十到上百周期——仍在内存,不碰硬盘。真正碰硬盘的是 page fault(缺页):数据根本不在内存、要从硬盘调入, $\sim$ 百万周期,慢几万倍。三层延迟阶梯:L1 ~ 4 周期 $\rightarrow$ DRAM/TLB walk $\sim$ 百周期 $\rightarrow$ 硬盘 $\sim$ 百万周期。顺序访问 TLB 常命中(一页 4KB=1024 个 int);随机大跨步 TLB 频繁 miss,叠加在数据 miss 之上。TLB 是每核一个的硬件,多进程共用,靠进程标签(ASID/PCID)区分——这也是 HFT pin 核隔离、避免上下文切换污染 TLB/cache 的动机之一。HFT 亦会锁定并预热内存(pre-fault),确保运行时永不缺页。

4. AoS vs SoA:数据布局如何决定 cache 有效带宽

100 万个粒子,每个有位置 (x,y,z) 、速度 (vx,vy,vz) 、以及一堆别的字段(mass、color、temp...),整个结构体假设 64 字节。有个热循环只更新位置:`x += vx; y += vy; z += vz;`只碰 6 个字段,不碰 mass/color/temp。两种布局:

- AoS(结构体数组):`Particle arr[N]`,内存里“一个粒子的所有字段挨在一起”。
- SoA(数组结构体):`float x[N], y[N], ..., vx[N], ...`,内存里“同一字段的所有粒子挨在一起”。

机制:cache line 的有效数据密度。只更新位置时,AoS 为拿粒子 0 的 `x`,cache 搬进整条 64 字节 = 整个粒子 0,但只用其中 24 字节(6 个位置/速度字段),剩 40 字节(mass/color/temp)白搬——每条 line 仅 38% 有效。SoA 访问 `x[]`,一条 line = 16 个粒子的 `x`,全部用得上;访问 `vx[]` 同理——每条 line 100% 有效。故处理同样的位置数据,SoA 的内存流量约为 AoS 的 38%,实测快 2–4 倍(理论上界 $\approx 100/38 \approx 2.6$,叠加 prefetch/SIMD 到 2–4)。原则:让一条 cache line 里装的,全是你马上要用的数据。

SoA 并非无脑更优,它有一个隐含前提:cache 要大到能同时容纳热循环需要的所有数组的当前 line(此处 6 条)。正常 cache(L1 就有几百条 line)轻松满足,故 SoA 通常赢。但把 cache 压到极端(比如只有 1 条

line), SoA 的 6 个字段各在不同 line、装不下、反复颠簸(thrashing), 反而是 AoS 赢(一个粒子的字段聚在一条 line, 1 条即可服务一个粒子)。判断法则: 批量处理少数字段 $\rightarrow$ SoA; 逐对象访问全部字段 $\rightarrow$ AoS; 两者之间用 AoSoA 折中(小块 AoS 装进一条 line, 块间 SoA)。列式存储在 ML/数值计算中无处不在, 正是 SoA 思想——对某个特征列做向量化运算时, 该列的所有值挨在一起, cache 与 SIMD 都吃满。

5. 内存墙与 Prefetch: 为何 Pointer Chasing 是性能杀手

一个数字先立在这: CPU 一个周期能做数次算术, 而一次 cache miss 到 DRAM 要 $\sim 100\text{--}300$ 周期——CPU 比内存快约 100 倍, 这道鸿沟叫内存墙。CPU 大部分时间在等内存, 不在算。遍历一个节点散落堆各处的链表(pointer chasing), 对每个节点做点计算:

```
Node* p = head;
while (p) {
    sum += compute(p->value); // a little computation
    p = p->next;              // jump to the next node (random address)
}
```

机制: 串行依赖链让内存延迟无法隐藏。这个循环的瓶颈是 $p=p->next$ 的访存, 不是 compute 的计算——计算几个周期就完, 而每跳到一个新节点要等一次 ~ 200 周期的 miss, CPU 大部分时间在 stall(停顿)、ALU 空转。prefetch 救不了它, 原因是一个“先有鸡还是先有蛋”的串行依赖: 要预取更远的节点, 得先知道其地址; 而“下下个”地址锁在“下一个节点的内容”里, 而下一个节点的内容还没搬回来。你手里只有紧邻的下一个地址(它与当前节点的 value 同在一条 line, 已命中), 但要提前很多步才能隐藏 200 周期的延迟——链表只能一步一步解锁地址, 每步串行等一次 miss, 提前不了。

对比数组为何能翻越内存墙: 数组的地址可提前计算($base+i*4$), 不依赖“上一次访存的内容”。于是 CPU 可以 prefetch(提前搬后续 line)+ MLP(同时发起多个 miss 让延迟重叠)。两者都能隐藏内存延迟。pointer chasing 两者皆失效: 地址算不出(prefetch 失效)、下一个地址依赖上一个的内容(MLP 失效)。所以链表遍历是 延迟 $\times$ 节点数 的纯串行等待, 数组是 延迟 $\div$ 并行度的重叠等待——这才是链表比数组慢一个数量级的根本原因, 不只是“节点散、miss 率高”, 而是串行依赖让延迟完全裸露、无法隐藏。

为什么 HFT 死守数组预分配——所有线索在此汇合。HFT 偏爱数组预分配、循环复用、热路径不 new, 有两层原因, 现在都串起来了。其一(回收层): 数组槽位复用、从不 delete, 绕过整个安全内存回收问题, 并顺带免疫 ABA(单调整数序号, 非复用指针)。其二(内存墙层, 本报告): 数组地址可预测 $\rightarrow$ prefetch/MLP 能翻越内存墙, 链表 pointer chasing 让内存延迟完全裸露, 翻不过去。“正确性”与“速度”两座山, 最终指向同一个工程结论: 能连续、能预测、能复用, 就别动态分配、别指针追逐。